

```

import os
import numpy as np
import tifffile as tiff
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F

from torch.utils.data import Dataset
from torch.utils.data import DataLoader
from torch.utils.data import random_split

from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay

from transformers import (
    SegformerForSemanticSegmentation
)

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Device:", device)

Device: cuda

root_dir =
"/kaggle/input/datasets/rishikeshgopal121/landslide/ReadyToBeUsedDataset"

class SegFormerDataset(Dataset):
    def __init__(self, root_dir):
        self.samples = []

        for region in os.listdir(
            os.path.join(root_dir, "6days")
        ):
            region_path = os.path.join(
                root_dir,
                "6days",
                region
            )

            folders = os.listdir(region_path)

```

```

for folder in folders:

    if folder == "Segmentations":
        continue

    coh_path = os.path.join(
        region_path,
        folder,
        "coherence"
    )

    phase_path = os.path.join(
        region_path,
        folder,
        "phase"
    )

    mask_path = os.path.join(
        region_path,
        "Segmentations",
        folder
    )

    if not (
        os.path.exists(coh_path)
        and
        os.path.exists(phase_path)
        and
        os.path.exists(mask_path)
    ):
        continue

    files = sorted(
        os.listdir(coh_path)
    )

    for file in files:

        if file.endswith(".tif"):

            self.samples.append(

                (

                    os.path.join(
                        coh_path,
                        file
                    ),

                    os.path.join(

```

```

        phase_path,
        file
    ),
    os.path.join(
        mask_path,
        file
    )
)
)

print(
    "Total Samples:",
    len(self.samples)
)

def __len__(self):
    return len(self.samples)

def __getitem__(self, idx):
    coh_file, phase_file, mask_file = self.samples[idx]

    coherence = tiff.imread(
        coh_file
    )

    phase = tiff.imread(
        phase_file
    )

    mask = tiff.imread(
        mask_file
    )

    cos_phase = np.cos(phase)
    sin_phase = np.sin(phase)
    image = np.stack(
        [
            coherence,
            cos_phase,
            sin_phase
        ],

```

```

        axis=0
    )

    mask = np.expand_dims(
        mask,
        axis=0
    )

    # ===== AUGMENTATION =====

    if np.random.rand() > 0.5:

        image = np.flip(
            image,
            axis=2
        ).copy()

        mask = np.flip(
            mask,
            axis=2
        ).copy()

    if np.random.rand() > 0.5:

        image = np.flip(
            image,
            axis=1
        ).copy()

        mask = np.flip(
            mask,
            axis=1
        ).copy()

    k = np.random.randint(0,4)

    image = np.rot90(
        image,
        k,
        axes=(1,2)
    ).copy()

    mask = np.rot90(
        mask,
        k,
        axes=(1,2)
    ).copy()

    return (

```

```

        torch.tensor(
            image,
            dtype=torch.float32
        ),

        torch.tensor(
            mask,
            dtype=torch.float32
        )
    )

dataset = SegFormerDataset(
    root_dir
)

Total Samples: 4655
train_size = int(0.70 * len(dataset))
val_size = int(0.15 * len(dataset))
test_size = len(dataset) - train_size - val_size
train_dataset, val_dataset, test_dataset = random_split(
    dataset,
    [
        train_size,
        val_size,
        test_size
    ]
)

print("Train:", len(train_dataset))
print("Validation:", len(val_dataset))
print("Test:", len(test_dataset))

Train: 3258
Validation: 698
Test: 699

train_loader = DataLoader(
    train_dataset,
    batch_size=8,
    shuffle=True
)

```

```

test_loader = DataLoader(
    test_dataset,
    batch_size=8,
    shuffle=False
)

val_loader = DataLoader(
    val_dataset,
    batch_size=8,
    shuffle=False
)

class SegFormerModel(nn.Module):
    def __init__(self):
        super().__init__()

        self.segformer =
SegformerForSemanticSegmentation.from_pretrained(
    "nvidia/segformer-b0-finetuned-ade-512-512",
    num_labels=1,
    ignore_mismatched_sizes=True
)

    def forward(self, x):
        outputs = self.segformer(
            pixel_values=x
        )

        logits = outputs.logits
        logits = F.interpolate(
            logits,
            size=(100, 100),
            mode="bilinear",

```

```

        align_corners=False
    )

    return logits

model = SegFormerModel().to(device)

print(model)

{"model_id": "58743d15555c48bcb462e7b5309f2fcf", "version_major": 2, "version_minor": 0}

```

SegformerForSemanticSegmentation LOAD REPORT from: nvidia/segformer-b0-finetuned-ade-512-512

Key	Status	
-----+-----		
+-----		

decode_head.classifier.bias	MISMATCH	Reinit due to size mismatch ckpt: torch.Size([150]) vs model:torch.Size([1])
decode_head.classifier.weight	MISMATCH	Reinit due to size mismatch ckpt: torch.Size([150, 256, 1, 1]) vs model:torch.Size([1, 256, 1, 1])

Notes:

- MISMATCH :ckpt weights were loaded, but they did not match the original empty weight shapes.

```

SegFormerModel(
  (segformer): SegformerForSemanticSegmentation(
    (segformer): SegformerModel(
      (encoder): SegformerEncoder(
        (patch_embeddings): ModuleList(
          (0): SegformerOverlapPatchEmbeddings(
            (proj): Conv2d(3, 32, kernel_size=(7, 7), stride=(4, 4),
padding=(3, 3))
            (layer_norm): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
          )
          (1): SegformerOverlapPatchEmbeddings(
            (proj): Conv2d(32, 64, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1))
            (layer_norm): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
          )
          (2): SegformerOverlapPatchEmbeddings(
            (proj): Conv2d(64, 160, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1))
            (layer_norm): LayerNorm((160,), eps=1e-05,

```

```

elementwise_affine=True)
    )
    (3): SegformerOverlapPatchEmbeddings(
      (proj): Conv2d(160, 256, kernel_size=(3, 3), stride=(2,
2), padding=(1, 1))
      (layer_norm): LayerNorm((256,)), eps=1e-05,
elementwise_affine=True)
    )
  )
  (block): ModuleList(
    (0): ModuleList(
      (0): SegformerLayer(
        (layer_norm_1): LayerNorm((32,)), eps=1e-05,
elementwise_affine=True)
        (attention): SegformerAttention(
          (self): SegformerEfficientSelfAttention(
            (query): Linear(in_features=32, out_features=32,
bias=True)
            (key): Linear(in_features=32, out_features=32,
bias=True)
            (value): Linear(in_features=32, out_features=32,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
            (sr): Conv2d(32, 32, kernel_size=(8, 8), stride=(8,
8))
            (layer_norm): LayerNorm((32,)), eps=1e-05,
elementwise_affine=True)
          )
          (output): SegformerSelfOutput(
            (dense): Linear(in_features=32, out_features=32,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
          )
        )
        (drop_path): Identity()
        (layer_norm_2): LayerNorm((32,)), eps=1e-05,
elementwise_affine=True)
        (mlp): SegformerMixFFN(
          (dense1): Linear(in_features=32, out_features=128,
bias=True)
          (dwconv): SegformerDWConv(
            (dwconv): Conv2d(128, 128, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=128)
          )
          (intermediate_act_fn): GELUActivation()
          (dense2): Linear(in_features=128, out_features=32,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
    )
  )
)

```



```

        )
        (1): SegformerLayer(
          (layer_norm_1): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
          (attention): SegformerAttention(
            (self): SegformerEfficientSelfAttention(
              (query): Linear(in_features=32, out_features=32,
bias=True)
              (key): Linear(in_features=32, out_features=32,
bias=True)
              (value): Linear(in_features=32, out_features=32,
bias=True)
              (dropout): Dropout(p=0.0, inplace=False)
              (sr): Conv2d(32, 32, kernel_size=(8, 8), stride=(8,
8))
              (layer_norm): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
            )
            (output): SegformerSelfOutput(
              (dense): Linear(in_features=32, out_features=32,
bias=True)
              (dropout): Dropout(p=0.0, inplace=False)
            )
          )
          (drop_path): SegformerDropPath(p=0.014285714365541935)
          (layer_norm_2): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
          (mlp): SegformerMixFFN(
            (dense1): Linear(in_features=32, out_features=128,
bias=True)
            (dwconv): SegformerDWConv(
              (dwconv): Conv2d(128, 128, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=128)
            )
            (intermediate_act_fn): GELUActivation()
            (dense2): Linear(in_features=128, out_features=32,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
          )
        )
      )
    (1): ModuleList(
      (0): SegformerLayer(
        (layer_norm_1): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
        (attention): SegformerAttention(
          (self): SegformerEfficientSelfAttention(
            (query): Linear(in_features=64, out_features=64,
bias=True)

```

```

        (key): Linear(in_features=64, out_features=64,
bias=True)
        (value): Linear(in_features=64, out_features=64,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
        (sr): Conv2d(64, 64, kernel_size=(4, 4), stride=(4,
4))
        (layer_norm): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
    )
    (output): SegformerSelfOutput(
        (dense): Linear(in_features=64, out_features=64,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
    )
    (drop_path): SegformerDropPath(p=0.02857142873108387)
    (layer_norm_2): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
    (mlp): SegformerMixFFN(
        (dense1): Linear(in_features=64, out_features=256,
bias=True)
        (dwconv): SegformerDWConv(
            (dwconv): Conv2d(256, 256, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=256)
        )
        (intermediate_act_fn): GELUActivation()
        (dense2): Linear(in_features=256, out_features=64,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
    )
    (1): SegformerLayer(
        (layer_norm_1): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
        (attention): SegformerAttention(
            (self): SegformerEfficientSelfAttention(
                (query): Linear(in_features=64, out_features=64,
bias=True)
                (key): Linear(in_features=64, out_features=64,
bias=True)
                (value): Linear(in_features=64, out_features=64,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
                (sr): Conv2d(64, 64, kernel_size=(4, 4), stride=(4,
4))
            )
            (layer_norm): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
        )
    )

```

```

        (output): SegformerSelfOutput(
          (dense): Linear(in_features=64, out_features=64,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): SegformerDropPath(p=0.04285714402794838)
      (layer_norm_2): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
      (mlp): SegformerMixFFN(
        (dense1): Linear(in_features=64, out_features=256,
bias=True)
        (dwconv): SegformerDWConv(
          (dwconv): Conv2d(256, 256, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=256)
        )
        (intermediate_act_fn): GELUActivation()
        (dense2): Linear(in_features=256, out_features=64,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
      )
    )
  )
  (2): ModuleList(
    (0): SegformerLayer(
      (layer_norm_1): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
      (attention): SegformerAttention(
        (self): SegformerEfficientSelfAttention(
          (query): Linear(in_features=160, out_features=160,
bias=True)
          (key): Linear(in_features=160, out_features=160,
bias=True)
          (value): Linear(in_features=160, out_features=160,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
          (sr): Conv2d(160, 160, kernel_size=(2, 2),
stride=(2, 2))
          (layer_norm): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
        )
        (output): SegformerSelfOutput(
          (dense): Linear(in_features=160, out_features=160,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): SegformerDropPath(p=0.05714285746216774)
      (layer_norm_2): LayerNorm((160,), eps=1e-05,

```

```

elementwise_affine=True)
    (mlp): SegformerMixFFN(
        (dense1): Linear(in_features=160, out_features=640,
bias=True)
        (dwconv): SegformerDWConv(
            (dwconv): Conv2d(640, 640, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=640)
            )
        (intermediate_act_fn): GELUActivation()
        (dense2): Linear(in_features=640, out_features=160,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
    )
)
(1): SegformerLayer(
    (layer_norm_1): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
    (attention): SegformerAttention(
        (self): SegformerEfficientSelfAttention(
            (query): Linear(in_features=160, out_features=160,
bias=True)
            (key): Linear(in_features=160, out_features=160,
bias=True)
            (value): Linear(in_features=160, out_features=160,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
            (sr): Conv2d(160, 160, kernel_size=(2, 2),
stride=(2, 2))
            (layer_norm): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
        )
        (output): SegformerSelfOutput(
            (dense): Linear(in_features=160, out_features=160,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
        )
    )
    (drop_path): SegformerDropPath(p=0.0714285746216774)
    (layer_norm_2): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
    (mlp): SegformerMixFFN(
        (dense1): Linear(in_features=160, out_features=640,
bias=True)
        (dwconv): SegformerDWConv(
            (dwconv): Conv2d(640, 640, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=640)
            )
        (intermediate_act_fn): GELUActivation()
        (dense2): Linear(in_features=640, out_features=160,

```

```

bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
    )
    )
    )
    (3): ModuleList(
      (0): SegformerLayer(
        (layer_norm_1): LayerNorm((256,), eps=1e-05,
elementwise_affine=True)
        (attention): SegformerAttention(
          (self): SegformerEfficientSelfAttention(
            (query): Linear(in_features=256, out_features=256,
bias=True)
            (key): Linear(in_features=256, out_features=256,
bias=True)
            (value): Linear(in_features=256, out_features=256,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
          )
          (output): SegformerSelfOutput(
            (dense): Linear(in_features=256, out_features=256,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
          )
        )
        (drop_path): SegformerDropPath(p=0.08571428805589676)
        (layer_norm_2): LayerNorm((256,), eps=1e-05,
elementwise_affine=True)
        (mlp): SegformerMixFFN(
          (dense1): Linear(in_features=256, out_features=1024,
bias=True)
          (dwconv): SegformerDWConv(
            (dwconv): Conv2d(1024, 1024, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=1024)
          )
          (intermediate_act_fn): GELUActivation()
          (dense2): Linear(in_features=1024, out_features=256,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
    )
    (1): SegformerLayer(
      (layer_norm_1): LayerNorm((256,), eps=1e-05,
elementwise_affine=True)
      (attention): SegformerAttention(
        (self): SegformerEfficientSelfAttention(
          (query): Linear(in_features=256, out_features=256,
bias=True)
          (key): Linear(in_features=256, out_features=256,

```

```

bias=True)
        (value): Linear(in_features=256, out_features=256,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
    )
    (output): SegformerSelfOutput(
        (dense): Linear(in_features=256, out_features=256,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
    )
    (drop_path): SegformerDropPath(p=0.10000000149011612)
    (layer_norm_2): LayerNorm((256,), eps=1e-05,
elementwise_affine=True)
    (mlp): SegformerMixFFN(
        (dense1): Linear(in_features=256, out_features=1024,
bias=True)
        (dwconv): SegformerDWConv(
            (dwconv): Conv2d(1024, 1024, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=1024)
        )
        (intermediate_act_fn): GELUActivation()
        (dense2): Linear(in_features=1024, out_features=256,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
    )
    )
    )
    (layer_norm): ModuleList(
      (0): LayerNorm((32,), eps=1e-05, elementwise_affine=True)
      (1): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
      (2): LayerNorm((160,), eps=1e-05, elementwise_affine=True)
      (3): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
    )
  )
  (decode_head): SegformerDecodeHead(
    (linear_c): ModuleList(
      (0): SegformerMLP(
        (proj): Linear(in_features=32, out_features=256, bias=True)
      )
      (1): SegformerMLP(
        (proj): Linear(in_features=64, out_features=256, bias=True)
      )
      (2): SegformerMLP(
        (proj): Linear(in_features=160, out_features=256, bias=True)
      )
      (3): SegformerMLP(

```

```

        (proj): Linear(in_features=256, out_features=256, bias=True)
    )
    (linear_fuse): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1,
1), bias=False)
    (batch_norm): BatchNorm2d(256, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
    (activation): ReLU()
    (dropout): Dropout(p=0.1, inplace=False)
    (classifier): Conv2d(256, 1, kernel_size=(1, 1), stride=(1, 1))
)
)
)
sample_batch = next(iter(train_loader))
images, masks = sample_batch
images = images.to(device)
with torch.no_grad():
    outputs = model(images)
print("Input Shape :", images.shape)
print("Output Shape:", outputs.shape)
Input Shape : torch.Size([8, 3, 100, 100])
Output Shape: torch.Size([8, 1, 100, 100])
class DiceLoss(nn.Module):
    def __init__(self):
        super().__init__()
    def forward(self, pred, target):
        pred = torch.sigmoid(pred)
        intersection = (
            pred * target
        ).sum()
        dice = (
            2 * intersection
        ) / (

```

```

        pred.sum()
        +
        target.sum()
        +
        1e-8

    )

    return 1 - dice

class FocalLoss(nn.Module):

    def __init__(
        self,
        alpha=0.8,
        gamma=2
    ):

        super().__init__()

        self.alpha = alpha

        self.gamma = gamma

    def forward(
        self,
        pred,
        target
    ):

        bce = F.binary_cross_entropy_with_logits(
            pred,
            target,
            reduction='none'
        )

        pt = torch.exp(-bce)

        focal = (
            self.alpha
            *
            (1 - pt) ** self.gamma

```



```

        *
        bce
    )

    return focal.mean()

dice_loss = DiceLoss()
focal_loss = FocalLoss()

def hybrid_loss(
    pred,
    target
):
    d = dice_loss(
        pred,
        target
    )

    f = focal_loss(
        pred,
        target
    )

    return d + f

class DiceLoss(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, pred, target):
        pred = torch.sigmoid(pred)

        intersection = (
            pred * target
        ).sum()

        dice = (
            2 * intersection
        ) / (

```

```

        pred.sum()
        +
        target.sum()
        +
        1e-8
    )

    return 1 - dice

class FocalLoss(nn.Module):
    def __init__(
        self,
        alpha=0.8,
        gamma=2
    ):
        super().__init__()

        self.alpha = alpha
        self.gamma = gamma

    def forward(
        self,
        pred,
        target
    ):
        bce = F.binary_cross_entropy_with_logits(
            pred,
            target,
            reduction='none'
        )

        pt = torch.exp(-bce)
        focal = (
            self.alpha
            *
            (1 - pt) ** self.gamma

```

```

        *
        bce
    )
    return focal.mean()

dice_loss = DiceLoss()
focal_loss = FocalLoss()

def hybrid_loss(
    pred,
    target
):
    d = dice_loss(
        pred,
        target
    )
    f = focal_loss(
        pred,
        target
    )
    return d + f

optimizer = optim.AdamW(
    model.parameters(),
    lr=0.0001,
    weight_decay=1e-4
)

scheduler = optim.lr_scheduler.ReduceLR0nPlateau(
    optimizer,
    mode='min',
    factor=0.5,
    patience=5
)

```

```

)
train_losses = []
train_acc = []
train_prec = []
train_rec = []
epochs = 120
best_loss = float('inf')
for epoch in range(epochs):
    model.train()
    total_loss = 0
    correct = 0
    total = 0
    TP = 0
    FP = 0
    FN = 0
    print(f"\nEpoch {epoch+1}/{epochs}")
    for i, (images, masks) in enumerate(train_loader):
        images = images.to(device)
        masks = masks.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = hybrid_loss(
            outputs,
            masks
        )
        loss.backward()

```

```
torch.nn.utils.clip_grad_norm_(
    model.parameters(),
    1.0
)
optimizer.step()
total_loss += loss.item()
# ===== METRICS =====
pred = torch.sigmoid(outputs)
pred_bin = (
    pred > 0.5
).float()
correct += (
    pred_bin == masks
).sum().item()
total += masks.numel()
TP += (
    pred_bin * masks
).sum().item()
FP += (
    pred_bin * (1 - masks)
).sum().item()
FN += (
    (1 - pred_bin) * masks
).sum().item()
progress = (
    (i+1)
```

```

        /
        len(train_loader)
    ) * 100
    print(
        f"\rProgress: {progress:.2f}% | Loss: {loss.item():.4f}",
        end=""
    )

# ===== EPOCH METRICS =====
acc = correct / total
precision = TP / (
    TP + FP + 1e-8
)
recall = TP / (
    TP + FN + 1e-8
)
train_losses.append(
    total_loss
)
train_acc.append(
    acc
)
train_prec.append(
    precision
)
train_rec.append(
    recall

```

```
)  
print(  
    f"\nLoss: {total_loss:.4f}"  
)  
print(  
    f"Accuracy : {acc:.4f}"  
)  
print(  
    f"Precision: {precision:.4f}"  
)  
print(  
    f"Recall    : {recall:.4f}"  
)  
scheduler.step(  
    total_loss  
)  
if total_loss < best_loss:  
    best_loss = total_loss  
    torch.save(  
        model.state_dict(),  
        "segformer_best.pth"  
    )  
    print(  
        "Model Saved!"  
    )
```

Epoch 1/120
Progress: 100.00% | Loss: 0.6307
Loss: 336.8664
Accuracy : 0.9531
Precision: 0.3752
Recall : 0.6332
Model Saved!

Epoch 2/120
Progress: 100.00% | Loss: 0.6093
Loss: 227.8648
Accuracy : 0.9757
Precision: 0.6285
Recall : 0.6426
Model Saved!

Epoch 3/120
Progress: 100.00% | Loss: 0.3356
Loss: 192.4844
Accuracy : 0.9781
Precision: 0.6722
Recall : 0.6547
Model Saved!

Epoch 4/120
Progress: 100.00% | Loss: 0.2471
Loss: 179.1829
Accuracy : 0.9792
Precision: 0.6969
Recall : 0.6533
Model Saved!

Epoch 5/120
Progress: 100.00% | Loss: 0.2508
Loss: 167.5047
Accuracy : 0.9804
Precision: 0.7191
Recall : 0.6681
Model Saved!

Epoch 6/120
Progress: 100.00% | Loss: 1.0461
Loss: 164.0867
Accuracy : 0.9808
Precision: 0.7199
Recall : 0.6840
Model Saved!

Epoch 7/120

Progress: 100.00% | Loss: 1.0882
Loss: 156.3077
Accuracy : 0.9818
Precision: 0.7357
Recall : 0.6983
Model Saved!

Epoch 8/120
Progress: 100.00% | Loss: 0.4263
Loss: 156.5566
Accuracy : 0.9814
Precision: 0.7267
Recall : 0.7004

Epoch 9/120
Progress: 100.00% | Loss: 0.2499
Loss: 150.2286
Accuracy : 0.9825
Precision: 0.7455
Recall : 0.7144
Model Saved!

Epoch 10/120
Progress: 100.00% | Loss: 0.1962
Loss: 147.1034
Accuracy : 0.9828
Precision: 0.7502
Recall : 0.7181
Model Saved!

Epoch 11/120
Progress: 100.00% | Loss: 1.0291
Loss: 139.9430
Accuracy : 0.9837
Precision: 0.7640
Recall : 0.7308
Model Saved!

Epoch 13/120
Progress: 100.00% | Loss: 0.2934
Loss: 139.7979
Accuracy : 0.9839
Precision: 0.7712
Recall : 0.7278
Model Saved!

Epoch 14/120
Progress: 100.00% | Loss: 1.0154
Loss: 133.6640
Accuracy : 0.9843

Precision: 0.7711
Recall : 0.7448
Model Saved!

Epoch 15/120
Progress: 100.00% | Loss: 0.3476
Loss: 133.3918
Accuracy : 0.9847
Precision: 0.7822
Recall : 0.7453
Model Saved!

Epoch 16/120
Progress: 100.00% | Loss: 0.4546
Loss: 131.4848
Accuracy : 0.9847
Precision: 0.7780
Recall : 0.7491
Model Saved!

Epoch 17/120
Progress: 100.00% | Loss: 0.1917
Loss: 129.8752
Accuracy : 0.9850
Precision: 0.7855
Recall : 0.7489
Model Saved!

Epoch 18/120
Progress: 100.00% | Loss: 0.4763
Loss: 125.6937
Accuracy : 0.9852
Precision: 0.7854
Recall : 0.7591
Model Saved!

Epoch 19/120
Progress: 100.00% | Loss: 0.4298
Loss: 124.9443
Accuracy : 0.9854
Precision: 0.7897
Recall : 0.7598
Model Saved!

Epoch 20/120
Progress: 100.00% | Loss: 0.3604
Loss: 124.4141
Accuracy : 0.9856
Precision: 0.7950
Recall : 0.7611

Model Saved!

Epoch 21/120

Progress: 100.00% | Loss: 0.0995

Loss: 120.6283

Accuracy : 0.9859

Precision: 0.7965

Recall : 0.7692

Model Saved!

Epoch 22/120

Progress: 100.00% | Loss: 0.2431

Loss: 120.3404

Accuracy : 0.9859

Precision: 0.7945

Recall : 0.7731

Model Saved!

Epoch 23/120

Progress: 100.00% | Loss: 1.0000

Loss: 119.2874

Accuracy : 0.9863

Precision: 0.8050

Recall : 0.7727

Model Saved!

Epoch 24/120

Progress: 100.00% | Loss: 1.0120

Loss: 113.7720

Accuracy : 0.9867

Precision: 0.8079

Recall : 0.7854

Model Saved!

Epoch 25/120

Progress: 100.00% | Loss: 0.1810

Loss: 115.0912

Accuracy : 0.9867

Precision: 0.8108

Recall : 0.7796

Epoch 26/120

Progress: 100.00% | Loss: 0.1399

Loss: 113.0992

Accuracy : 0.9869

Precision: 0.8127

Recall : 0.7850

Model Saved!

Epoch 27/120

Progress: 100.00% | Loss: 0.2954
Loss: 110.1154
Accuracy : 0.9874
Precision: 0.8175
Recall : 0.7943
Model Saved!

Epoch 28/120
Progress: 100.00% | Loss: 0.2840
Loss: 110.5295
Accuracy : 0.9871
Precision: 0.8136
Recall : 0.7911

Epoch 29/120
Progress: 100.00% | Loss: 1.0000
Loss: 109.5083
Accuracy : 0.9873
Precision: 0.8188
Recall : 0.7913
Model Saved!

Epoch 30/120
Progress: 100.00% | Loss: 0.1378
Loss: 106.0776
Accuracy : 0.9874
Precision: 0.8175
Recall : 0.7973
Model Saved!

Epoch 31/120
Progress: 100.00% | Loss: 0.1994
Loss: 103.6237
Accuracy : 0.9878
Precision: 0.8237
Recall : 0.8005
Model Saved!

Epoch 32/120
Progress: 100.00% | Loss: 0.6821
Loss: 102.2801
Accuracy : 0.9881
Precision: 0.8278
Recall : 0.8063
Model Saved!

Epoch 33/120
Progress: 100.00% | Loss: 0.2319
Loss: 103.3860
Accuracy : 0.9881

Precision: 0.8309
Recall : 0.8018

Epoch 34/120
Progress: 100.00% | Loss: 1.1248
Loss: 102.3540
Accuracy : 0.9882
Precision: 0.8293
Recall : 0.8074

Epoch 35/120
Progress: 100.00% | Loss: 0.1377
Loss: 100.1385
Accuracy : 0.9884
Precision: 0.8321
Recall : 0.8130
Model Saved!

Epoch 36/120
Progress: 100.00% | Loss: 0.1871
Loss: 99.0603
Accuracy : 0.9884
Precision: 0.8307
Recall : 0.8134
Model Saved!

Epoch 37/120
Progress: 100.00% | Loss: 0.2972
Loss: 98.8168
Accuracy : 0.9885
Precision: 0.8355
Recall : 0.8124
Model Saved!

Epoch 38/120
Progress: 100.00% | Loss: 1.0000
Loss: 99.2666
Accuracy : 0.9888
Precision: 0.8377
Recall : 0.8187

Epoch 39/120
Progress: 100.00% | Loss: 0.3093
Loss: 96.6551
Accuracy : 0.9887
Precision: 0.8351
Recall : 0.8190
Model Saved!

Epoch 40/120

Progress: 100.00% | Loss: 0.1061
Loss: 94.8051
Accuracy : 0.9890
Precision: 0.8438
Recall : 0.8171
Model Saved!

Epoch 41/120
Progress: 100.00% | Loss: 0.1706
Loss: 95.8192
Accuracy : 0.9890
Precision: 0.8418
Recall : 0.8208

Epoch 42/120
Progress: 100.00% | Loss: 0.1250
Loss: 93.3644
Accuracy : 0.9892
Precision: 0.8452
Recall : 0.8240
Model Saved!

Epoch 43/120
Progress: 100.00% | Loss: 1.0566
Loss: 89.9606
Accuracy : 0.9894
Precision: 0.8459
Recall : 0.8316
Model Saved!

Epoch 44/120
Progress: 100.00% | Loss: 0.3121
Loss: 89.4280
Accuracy : 0.9897
Precision: 0.8497
Recall : 0.8366
Model Saved!

Epoch 45/120
Progress: 100.00% | Loss: 0.1027
Loss: 91.2159
Accuracy : 0.9893
Precision: 0.8424
Recall : 0.8323

Epoch 46/120
Progress: 100.00% | Loss: 0.1119
Loss: 89.4157
Accuracy : 0.9895
Precision: 0.8476

Recall : 0.8304
Model Saved!

Epoch 47/120
Progress: 100.00% | Loss: 0.2297
Loss: 88.6357
Accuracy : 0.9897
Precision: 0.8525
Recall : 0.8303
Model Saved!

Epoch 48/120
Progress: 100.00% | Loss: 0.1861
Loss: 87.3630
Accuracy : 0.9898
Precision: 0.8527
Recall : 0.8356
Model Saved!

Epoch 49/120
Progress: 100.00% | Loss: 0.1836
Loss: 86.7341
Accuracy : 0.9900
Precision: 0.8554
Recall : 0.8376
Model Saved!

Epoch 50/120
Progress: 100.00% | Loss: 0.1585
Loss: 84.8962
Accuracy : 0.9900
Precision: 0.8533
Recall : 0.8418
Model Saved!

Epoch 51/120
Progress: 100.00% | Loss: 0.2268
Loss: 84.7669
Accuracy : 0.9901
Precision: 0.8562
Recall : 0.8399
Model Saved!

Epoch 52/120
Progress: 100.00% | Loss: 1.0000
Loss: 84.4934
Accuracy : 0.9902
Precision: 0.8571
Recall : 0.8428
Model Saved!

Epoch 53/120
Progress: 100.00% | Loss: 0.1064
Loss: 83.1023
Accuracy : 0.9902
Precision: 0.8547
Recall : 0.8455
Model Saved!

Epoch 54/120
Progress: 100.00% | Loss: 0.3783
Loss: 82.7533
Accuracy : 0.9903
Precision: 0.8590
Recall : 0.8458
Model Saved!

Epoch 55/120
Progress: 100.00% | Loss: 0.1753
Loss: 83.5195
Accuracy : 0.9903
Precision: 0.8589
Recall : 0.8453

Epoch 56/120
Progress: 100.00% | Loss: 0.3150
Loss: 83.8725
Accuracy : 0.9904
Precision: 0.8579
Recall : 0.8501

Epoch 57/120
Progress: 100.00% | Loss: 0.0966
Loss: 79.0426
Accuracy : 0.9906
Precision: 0.8624
Recall : 0.8519
Model Saved!

Epoch 58/120
Progress: 100.00% | Loss: 0.3008
Loss: 78.2953
Accuracy : 0.9906
Precision: 0.8619
Recall : 0.8514
Model Saved!

Epoch 59/120
Progress: 100.00% | Loss: 0.1377
Loss: 79.4898

Accuracy : 0.9906
Precision: 0.8618
Recall : 0.8509

Epoch 60/120
Progress: 100.00% | Loss: 0.1071
Loss: 80.1146
Accuracy : 0.9907
Precision: 0.8643
Recall : 0.8517

Epoch 61/120
Progress: 100.00% | Loss: 0.5763
Loss: 79.5881
Accuracy : 0.9908
Precision: 0.8651
Recall : 0.8546

Epoch 62/120
Progress: 100.00% | Loss: 0.0949
Loss: 77.0560
Accuracy : 0.9908
Precision: 0.8652
Recall : 0.8550
Model Saved!

Epoch 63/120
Progress: 100.00% | Loss: 0.1114
Loss: 77.5548
Accuracy : 0.9908
Precision: 0.8672
Recall : 0.8533

Epoch 64/120
Progress: 100.00% | Loss: 0.3310
Loss: 76.2164
Accuracy : 0.9909
Precision: 0.8668
Recall : 0.8573
Model Saved!

Epoch 65/120
Progress: 100.00% | Loss: 0.1918
Loss: 75.6652
Accuracy : 0.9910
Precision: 0.8682
Recall : 0.8574
Model Saved!

Epoch 66/120

Progress: 100.00% | Loss: 0.1285
Loss: 73.3517
Accuracy : 0.9912
Precision: 0.8720
Recall : 0.8608
Model Saved!

Epoch 67/120
Progress: 100.00% | Loss: 1.0000
Loss: 75.9675
Accuracy : 0.9910
Precision: 0.8678
Recall : 0.8599

Epoch 68/120
Progress: 100.00% | Loss: 0.2173
Loss: 75.2737
Accuracy : 0.9911
Precision: 0.8718
Recall : 0.8579

Epoch 69/120
Progress: 100.00% | Loss: 0.1230
Loss: 75.1296
Accuracy : 0.9911
Precision: 0.8688
Recall : 0.8621

Epoch 70/120
Progress: 100.00% | Loss: 1.0000
Loss: 73.0766
Accuracy : 0.9914
Precision: 0.8739
Recall : 0.8658
Model Saved!

Epoch 71/120
Progress: 100.00% | Loss: 0.2789
Loss: 72.0386
Accuracy : 0.9913
Precision: 0.8724
Recall : 0.8635
Model Saved!

Epoch 72/120
Progress: 100.00% | Loss: 0.1844
Loss: 74.1110
Accuracy : 0.9913
Precision: 0.8712
Recall : 0.8634

Epoch 73/120
Progress: 100.00% | Loss: 0.1775
Loss: 73.0540
Accuracy : 0.9914
Precision: 0.8738
Recall : 0.8639

Epoch 74/120
Progress: 100.00% | Loss: 0.2299
Loss: 70.1920
Accuracy : 0.9916
Precision: 0.8767
Recall : 0.8678
Model Saved!

Epoch 75/120
Progress: 100.00% | Loss: 0.1520
Loss: 71.4733
Accuracy : 0.9915
Precision: 0.8760
Recall : 0.8655

Epoch 76/120
Progress: 100.00% | Loss: 0.1236
Loss: 70.9137
Accuracy : 0.9917
Precision: 0.8777
Recall : 0.8688

Epoch 77/120
Progress: 100.00% | Loss: 0.1873
Loss: 70.8314
Accuracy : 0.9917
Precision: 0.8760
Recall : 0.8729

Epoch 78/120
Progress: 100.00% | Loss: 0.1403
Loss: 69.7466
Accuracy : 0.9917
Precision: 0.8761
Recall : 0.8708
Model Saved!

Epoch 79/120
Progress: 100.00% | Loss: 0.2713
Loss: 68.8039
Accuracy : 0.9917
Precision: 0.8767

Recall : 0.8717
Model Saved!

Epoch 80/120
Progress: 100.00% | Loss: 1.0000
Loss: 69.2987
Accuracy : 0.9919
Precision: 0.8798
Recall : 0.8724

Epoch 81/120
Progress: 100.00% | Loss: 0.0859
Loss: 68.9757
Accuracy : 0.9919
Precision: 0.8782
Recall : 0.8751

Epoch 82/120
Progress: 100.00% | Loss: 0.2596
Loss: 67.7424
Accuracy : 0.9919
Precision: 0.8788
Recall : 0.8769
Model Saved!

Epoch 83/120
Progress: 100.00% | Loss: 0.1134
Loss: 67.6725
Accuracy : 0.9920
Precision: 0.8814
Recall : 0.8757
Model Saved!

Epoch 84/120
Progress: 100.00% | Loss: 0.2581
Loss: 67.3707
Accuracy : 0.9920
Precision: 0.8814
Recall : 0.8746
Model Saved!

Epoch 85/120
Progress: 100.00% | Loss: 0.1344
Loss: 66.9113
Accuracy : 0.9920
Precision: 0.8831
Recall : 0.8741
Model Saved!

Epoch 86/120

Progress: 100.00% | Loss: 1.0000
Loss: 68.0290
Accuracy : 0.9922
Precision: 0.8826
Recall : 0.8794

Epoch 87/120
Progress: 100.00% | Loss: 0.2071
Loss: 65.9292
Accuracy : 0.9921
Precision: 0.8832
Recall : 0.8774
Model Saved!

Epoch 88/120
Progress: 100.00% | Loss: 0.0875
Loss: 65.2956
Accuracy : 0.9922
Precision: 0.8864
Recall : 0.8756
Model Saved!

Epoch 89/120
Progress: 100.00% | Loss: 0.1236
Loss: 66.8763
Accuracy : 0.9920
Precision: 0.8833
Recall : 0.8742

Epoch 90/120
Progress: 100.00% | Loss: 0.2502
Loss: 65.4950
Accuracy : 0.9923
Precision: 0.8852
Recall : 0.8798

Epoch 91/120
Progress: 100.00% | Loss: 0.1010
Loss: 64.6693
Accuracy : 0.9923
Precision: 0.8843
Recall : 0.8809
Model Saved!

Epoch 92/120
Progress: 100.00% | Loss: 0.1669
Loss: 64.6421
Accuracy : 0.9924
Precision: 0.8878
Recall : 0.8822

Model Saved!

Epoch 93/120

Progress: 100.00% | Loss: 0.1282

Loss: 63.5376

Accuracy : 0.9924

Precision: 0.8865

Recall : 0.8813

Model Saved!

Epoch 94/120

Progress: 100.00% | Loss: 0.1187

Loss: 63.2696

Accuracy : 0.9924

Precision: 0.8865

Recall : 0.8821

Model Saved!

Epoch 95/120

Progress: 100.00% | Loss: 1.0000

Loss: 63.5612

Accuracy : 0.9925

Precision: 0.8881

Recall : 0.8834

Epoch 96/120

Progress: 100.00% | Loss: 0.2360

Loss: 63.2350

Accuracy : 0.9924

Precision: 0.8869

Recall : 0.8839

Model Saved!

Epoch 97/120

Progress: 100.00% | Loss: 0.2650

Loss: 62.1103

Accuracy : 0.9926

Precision: 0.8883

Recall : 0.8863

Model Saved!

Epoch 98/120

Progress: 100.00% | Loss: 0.3198

Loss: 62.5318

Accuracy : 0.9926

Precision: 0.8904

Recall : 0.8834

Epoch 99/120

Progress: 100.00% | Loss: 0.1746

Loss: 62.4981
Accuracy : 0.9925
Precision: 0.8883
Recall : 0.8850

Epoch 100/120
Progress: 100.00% | Loss: 0.0733
Loss: 62.6521
Accuracy : 0.9926
Precision: 0.8896
Recall : 0.8842

Epoch 101/120
Progress: 100.00% | Loss: 0.1715
Loss: 63.1794
Accuracy : 0.9925
Precision: 0.8879
Recall : 0.8844

Epoch 102/120
Progress: 100.00% | Loss: 0.1142
Loss: 61.5093
Accuracy : 0.9926
Precision: 0.8907
Recall : 0.8858
Model Saved!

Epoch 103/120
Progress: 100.00% | Loss: 0.0800
Loss: 59.3155
Accuracy : 0.9928
Precision: 0.8922
Recall : 0.8890
Model Saved!

Epoch 104/120
Progress: 100.00% | Loss: 0.2764
Loss: 61.6310
Accuracy : 0.9928
Precision: 0.8914
Recall : 0.8897

Epoch 105/120
Progress: 100.00% | Loss: 1.0000
Loss: 60.1102
Accuracy : 0.9929
Precision: 0.8935
Recall : 0.8910

Epoch 106/120

Progress: 100.00% | Loss: 0.1930
Loss: 59.6381
Accuracy : 0.9928
Precision: 0.8917
Recall : 0.8907

Epoch 107/120
Progress: 100.00% | Loss: 0.1184
Loss: 61.0279
Accuracy : 0.9928
Precision: 0.8928
Recall : 0.8898

Epoch 108/120
Progress: 100.00% | Loss: 0.1655
Loss: 58.1498
Accuracy : 0.9929
Precision: 0.8928
Recall : 0.8935
Model Saved!

Epoch 109/120
Progress: 100.00% | Loss: 0.3148
Loss: 59.0466
Accuracy : 0.9930
Precision: 0.8946
Recall : 0.8928

Epoch 110/120
Progress: 100.00% | Loss: 0.1829
Loss: 59.7582
Accuracy : 0.9929
Precision: 0.8928
Recall : 0.8907

Epoch 111/120
Progress: 100.00% | Loss: 0.1133
Loss: 57.6796
Accuracy : 0.9930
Precision: 0.8947
Recall : 0.8946
Model Saved!

Epoch 112/120
Progress: 100.00% | Loss: 0.2614
Loss: 57.7845
Accuracy : 0.9930
Precision: 0.8952
Recall : 0.8928

Epoch 113/120
Progress: 100.00% | Loss: 0.1589
Loss: 59.1616
Accuracy : 0.9931
Precision: 0.8962
Recall : 0.8958

Epoch 114/120
Progress: 100.00% | Loss: 0.1090
Loss: 59.0574
Accuracy : 0.9930
Precision: 0.8946
Recall : 0.8925

Epoch 115/120
Progress: 100.00% | Loss: 0.1310
Loss: 57.0542
Accuracy : 0.9932
Precision: 0.8987
Recall : 0.8960
Model Saved!

Epoch 116/120
Progress: 100.00% | Loss: 0.1537
Loss: 56.9842
Accuracy : 0.9931
Precision: 0.8967
Recall : 0.8948
Model Saved!

Epoch 117/120
Progress: 100.00% | Loss: 0.1291
Loss: 57.7897
Accuracy : 0.9931
Precision: 0.8969
Recall : 0.8922

Epoch 118/120
Progress: 100.00% | Loss: 0.1449
Loss: 57.2464
Accuracy : 0.9931
Precision: 0.8952
Recall : 0.8974

Epoch 119/120
Progress: 100.00% | Loss: 0.1946
Loss: 58.1002
Accuracy : 0.9931
Precision: 0.8967
Recall : 0.8948

Epoch 120/120
Progress: 100.00% | Loss: 1.0511
Loss: 56.7758
Accuracy : 0.9932
Precision: 0.8988
Recall : 0.8955
Model Saved!

```
def calculate_all_metrics(pred, target):
```

```
    pred = (pred > 0.5).float()
```

```
    TP = (pred * target).sum()
```

```
    TN = ((1-pred) * (1-target)).sum()
```

```
    FP = (pred * (1-target)).sum()
```

```
    FN = ((1-pred) * target).sum()
```

```
    accuracy = (
```

```
        TP + TN
```

```
    ) / (
```

```
        TP + TN + FP + FN + 1e-8
```

```
    )
```

```
    precision = TP / (
```

```
        TP + FP + 1e-8
```

```
    )
```

```
    recall = TP / (
```

```
        TP + FN + 1e-8
```

```
    )
```

```
    f1 = (
```

```
        2 * TP
```

```
    ) / (
```

```
        2 * TP + FP + FN + 1e-8
```

```
    )
```

```

    dice = (
        2 * TP
    ) / (
        2 * TP + FP + FN + 1e-8
    )
    iou = TP / (
        TP + FP + FN + 1e-8
    )
    return (
        accuracy.item(),
        precision.item(),
        recall.item(),
        f1.item(),
        dice.item(),
        iou.item()
    )
model.load_state_dict(
    torch.load(
        "segformer_best.pth"
    )
)
model.eval()
acc = 0
prec = 0
rec = 0
f1_total = 0
dice_total = 0

```

```
iou_total = 0
count = 0
all_preds = []
all_targets = []
for images, masks in test_loader:
    images = images.to(device)
    masks = masks.to(device)
    with torch.no_grad():
        pred = model(images)
        pred = torch.sigmoid(pred)
        a, p, r, f1, d, i = calculate_all_metrics(
            pred,
            masks
        )
        acc += a
        prec += p
        rec += r
        f1_total += f1
        dice_total += d
        iou_total += i
        count += 1
        pred_bin = (
            pred > 0.5
        ).cpu().numpy().flatten()
        target_bin = masks.cpu().numpy().flatten()
        all_preds.extend(pred_bin)
```

```

        all_targets.extend(target_bin)
print("\nFINAL RESULTS\n")
print("Accuracy :", acc/count)
print("Precision:", prec/count)
print("Recall   :", rec/count)
print("F1 Score :", f1_total/count)
print("Dice     :", dice_total/count)
print("IoU      :", iou_total/count)
cm = confusion_matrix(
    all_targets,
    all_preds
)
ConfusionMatrixDisplay(cm).plot()
plt.title(
    "SegFormer Confusion Matrix"
)
plt.show()

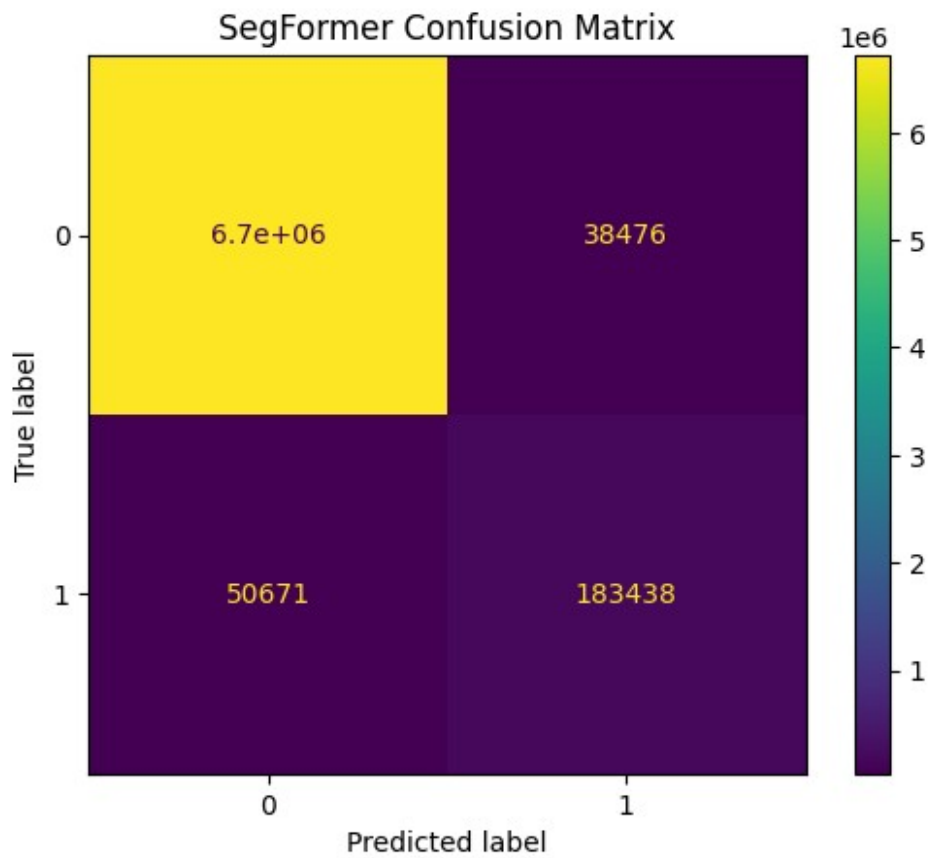
```

FINAL RESULTS

```

Accuracy : 0.9872622645714066
Precision: 0.8047757077623497
Recall   : 0.7574983914806084
F1 Score : 0.7651057940992442
Dice     : 0.7651057940992442
IoU      : 0.6355449364266612

```



===== VALIDATION RESULTS =====

```
val_accuracy = acc / count
val_precision = prec / count
val_recall = rec / count
val_f1 = f1_total / count
val_dice = dice_total / count
val_iou = iou_total / count
```

```
print("\nVALIDATION RESULTS\n")
```

```
print("Validation Accuracy :", val_accuracy)
print("Validation Precision:", val_precision)
print("Validation Recall   :", val_recall)
print("Validation F1 Score  :", val_f1)
print("Validation Dice      :", val_dice)
print("Validation IoU       :", val_iou)
```

VALIDATION RESULTS

```
Validation Accuracy : 0.9872622645714066
Validation Precision: 0.8047757077623497
```

```
Validation Recall    : 0.7574983914806084
Validation F1 Score  : 0.7651057940992442
Validation Dice      : 0.7651057940992442
Validation IoU       : 0.6355449364266612
```

```
epochs_range = range(
    1,
    len(train_losses)+1
)

plt.figure(figsize=(15,10))

# ===== LOSS =====

plt.subplot(2,2,1)

plt.plot(
    epochs_range,
    train_losses,
    marker='o'
)

plt.title(
    "Loss vs Epoch"
)

plt.grid()

# ===== ACCURACY =====

plt.subplot(2,2,2)

plt.plot(
    epochs_range,
    train_acc,
    marker='o'
)

plt.title(
```

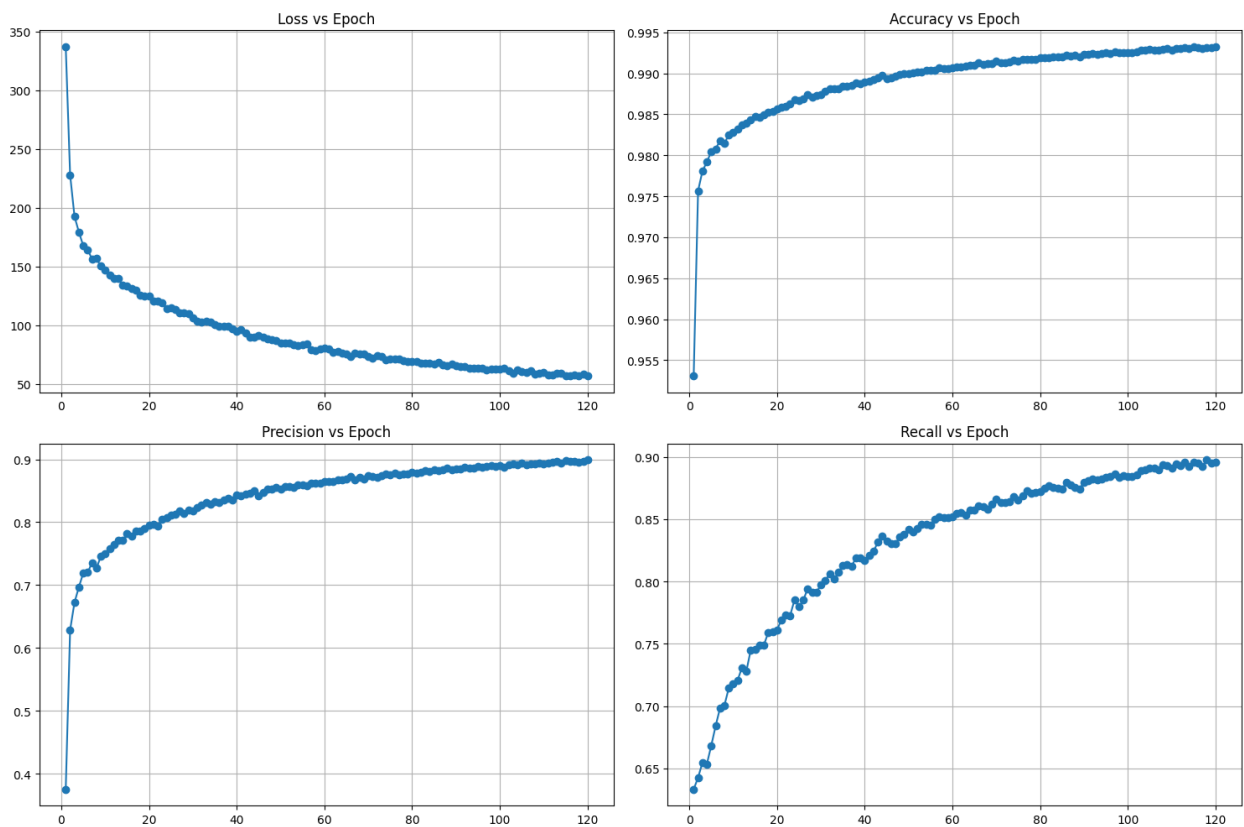
```

        "Accuracy vs Epoch"
    )
plt.grid()
# ===== PRECISION =====
plt.subplot(2,2,3)
plt.plot(
    epochs_range,
    train_prec,
    marker='o'
)
plt.title(
    "Precision vs Epoch"
)
plt.grid()
# ===== RECALL =====
plt.subplot(2,2,4)
plt.plot(
    epochs_range,
    train_rec,
    marker='o'
)
plt.title(
    "Recall vs Epoch"
)
plt.grid()
plt.tight_layout()

```



```
plt.show()
```



```
model.eval()

images_collected = 0
num_images = 10
all_inputs = []
all_masks = []
all_preds = []

for images, masks in test_loader:
    images = images.to(device)
    with torch.no_grad():
        pred = model(images)
        pred = torch.sigmoid(pred)
```

```
all_inputs.extend(
    images.cpu().numpy()
)
all_masks.extend(
    masks.numpy()
)
all_preds.extend(
    pred.cpu().numpy()
)
images_collected += images.shape[0]
if images_collected >= num_images:
    break

all_inputs = np.array(all_inputs)
all_masks = np.array(all_masks)
all_preds = np.array(all_preds)
plt.figure(figsize=(20,30))
for i in range(num_images):
    input_img = all_inputs[i][0]
    true_mask = all_masks[i][0]
    pred_mask = all_preds[i][0]
    binary_pred = (
        pred_mask > 0.5
    ).astype(float)
    heatmap = pred_mask
    overlay = (
        0.6 * input_img
```

```
        +
        0.4 * heatmap
    )
    # INPUT
    plt.subplot(
        num_images,
        5,
        i*5 + 1
    )
    plt.title("Input")
    plt.imshow(
        input_img,
        cmap='gray'
    )
    plt.axis('off')
    # GROUND TRUTH
    plt.subplot(
        num_images,
        5,
        i*5 + 2
    )
    plt.title("Ground Truth")
    plt.imshow(
        true_mask,
        cmap='gray'
    )
```

```
plt.axis('off')
# PREDICTION
plt.subplot(
    num_images,
    5,
    i*5 + 3
)
plt.title("Prediction")
plt.imshow(
    binary_pred,
    cmap='gray'
)
plt.axis('off')
# HEATMAP
plt.subplot(
    num_images,
    5,
    i*5 + 4
)
plt.title("Heatmap")
plt.imshow(
    heatmap,
    cmap='jet'
)
plt.axis('off')
# OVERLAY
```

```
plt.subplot(  
    num_images,  
    5,  
    i*5 + 5  
)  
plt.title("Overlay")  
plt.imshow(  
    overlay,  
    cmap='jet'  
)  
plt.axis('off')  
plt.tight_layout()  
plt.show()
```

